C-Sign: Signed Messages for Stellar Contract Accounts

Project overview · updated 27 October 2026 · Seyit (@sayweer)

Status	Working on testnet: verifier deployed, TypeScript library, 25 testnet experiments passing, live demo.
Live demo	c-sign-demo.vercel.app (testnet, passkey sign-in without a transaction)
Repository	github.com/sayweer/c-sign
Network	Stellar testnet first, mainnet later
Draft SEP	"Signed Messages for Contract Accounts", the counterpart of SEP-53 for contract accounts

What it is

C-Sign lets any Stellar contract account (a `C…` address: passkey wallets, multisig accounts, agent wallets with spending policies) sign a human-readable message, and lets anyone verify that signature against the account's **current** on-chain rules, without sending a transaction and without paying a fee.

It is Stellar's counterpart to Ethereum's ERC-1271, with one advantage: wallets do not have to write any code per account, because Stellar contract accounts already enforce their rules on-chain.

The project has three parts: a draft SEP, a tiny reference verifier contract that anyone can deploy, and a TypeScript library that wallets and servers can use.

The problem

Stellar has two kinds of accounts:

- **Classic accounts (`G…`)** are bound to one keypair. SEP-53 defines how they sign a message and how anyone verifies it.
- **Contract accounts (`C…`, "smart accounts")** have no single key. Who may act, with which threshold and under which policy, is decided by the account's own `__check_auth` code on-chain.

There is no standard way for a contract account to say "I signed this message". SEP-53 covers only `G…` keys, and the wallet interface standard SEP-43 leaves `signMessage` undefined for `C…` addresses.

In practice this breaks in two ways:

1. **Wallets throw.** jes-labs, Latch and Pollar all return an error when `signMessage` is called on a contract account.
2. **Wallets invent their own envelope.** SoroPass, Nido and Veridex each return a different raw WebAuthn assertion. These formats are mutually incompatible, and they can only be checked against a single public key, not against the account's rules. A key that was removed from the account still produces a "valid" signature.

Who is asking for it

Who	Where	Date	What they say
SDF, Jake Urban (SEP-43 author)	stellar-protocol#1928	2026-05-06	SEP-43 "should explicitly call out that the behavior is undefined for the contract account case (or alternatively provide an approach)"
SoroPass (SCF #44, Passkey UI Kit)	Stellar Wallets Kit #95	2026-06-17	"signMessage on a contract account has no standard on-chain verification. Should a module reject it, or do you have a preferred convention?"
jes-labs	Stellar Wallets Kit #93	2026-06-12	"signMessage throws … arbitrary message signing has no standard meaning for a contract account"
Latch (SCF #41)	README	2026-09-24	"`signAuthEntry` and `signMessage` fail explicitly because those provider methods are not available yet"
Pollar (`@pollar/core`, SCF #45)	npm README	2026-08-24	"smart (passkey) wallets yield an error outcome — a C-address has no classic ed25519 key to prove"
SoroPass, Nido, Veridex	SWK PR #112, Nido `walletSign.ts`, SWK #90	2026	Three incompatible ad-hoc WebAuthn envelopes; "verification is application-defined for now"

Where the design stands with SDF

On 24 September 2026 I posted the C-Sign design as a comment on stellar-protocol#1928, taking up the "provide an approach" option. Jake Urban (SDF) replied the same night:

> "Hey @sayweer, thanks for taking interest in this topic. Lets move discussion about the potential solution for contract-account message signing to a different github issue. Can you create one, include the description of the solution you just proposed, and reference this issue so its linked?"

SDF did not close the question or point to an existing solution. I opened the dedicated issue, stellar-protocol#2027, the next day. Jake replied there on 25 September:

- The outcome should be **a new SEP**, the counterpart of SEP-53, with a **separate SEP-43 change** proposed in parallel and merged once the new SEP is Final.
- There should be **no "canonical" verifier contract**; wallets should be free to choose who owns the verifier.
- He asked the SEP-45 authors (Leigh McCulloch and Marcelo Salloum) to weigh in on the nonce question.

On 20 October I posted the testnet results to #2027, including the answer to the nonce question: a signature submitted on-chain fails and leaves the nonce unused. On 22 October I moved the SEP discussion to stellar-protocol Discussion #2045, the step new SEPs take before a pull request. Eunomia, an agent treasury on Soroban, replied there that it would likely be an early user.

The design in this repository has been updated accordingly: docs/SPEC.md is now written as a new SEP following the SEP template, the verifier is trusted by its code hash instead of a fixed address, and the SEP-43 change lives in its own document. After a threat model (THREAT-MODEL.md) the specification is at version 0.0.2.

How it works

C-Sign attaches message signing to the authorization system Stellar already has.

1. **Readable message.** The dapp asks for a statement, for example `"Proposal #12 = YES"`, with a one-time nonce. The **wallet** adds the dapp's domain from the requesting origin, so a phishing site cannot obtain a signature addressed to another site. The message is a small map (`domain`, `statement`, `nonce`, `issued_at`, `version`) that the wallet shows as plain text. Nothing is signed blind.
2. **Signing.** The wallet signs a `SorobanAuthorizationEntry` whose root call is `verify_message(account, message)` on an instance of a small, admin-less reference verifier contract, with no sub-calls. Network id, nonce and expiration are already inside the signed preimage, and V2 address credentials (CAP-71) bind the signature to one account.
3. **No side effects.** `verify_message` runs `account.require_auth()` and then always fails with a reserved error code, `AUTH_OK`. If someone submits a published signature on-chain, the transaction fails and the account's nonce is not consumed.
4. **Verification.** The relying party rebuilds the expected call, checks that the verifier instance runs the reference code (by its Wasm hash), then runs `simulateTransaction` with `authMode: "enforce"` and accepts only `AUTH_OK` raised by that instance. The network executes the account's own `__check_auth`: are the signers still authorized, is the threshold met, does the policy allow it? No transaction is submitted, no fee is paid, and the answer reflects the account's rules **today**. Replay protection off-chain comes from the relying party's one-time `nonce`, as in Sign-In With Ethereum.

No canonical address. Anyone can deploy the reference verifier. Wallets and relying parties trust an instance only if its executable hash matches the published one. Without that check, someone could deploy a fake "verifier" that returns the success code without asking the account at all; with it, ownership of the contract stops mattering.

Why this shape works on Stellar

- **No work for wallets.** On Ethereum every smart wallet must implement `isValidSignature` (ERC-1271). On Stellar every contract account already enforces its rules in `__check_auth`, so C-Sign works with existing accounts without upgrades.
- **A proven pattern.** SEP-45 already authenticates contract accounts for web sessions with exactly this mechanism, an authorization entry plus enforcing simulation. C-Sign generalizes it to arbitrary messages and removes the server-side signature.
- **The host guarantees the answer.** The Soroban host converts every `__check_auth` failure into `Error(Auth, InvalidAction)`, so an account cannot forge the `AUTH_OK` code. The RPC error response exposes the code and diagnostic events. Required versions: stellar-rpc 23 or newer, `SimulationAuthMode` in js-stellar-sdk.

What C-Sign is not

- Not a login or account-recovery product. No Google or e-mail sign-in, no guardians, no signer management.
- No zero-knowledge proofs. None are needed.
- Holds no funds, needs no license.

What exists when C-Sign is done

1. **A standard.** "Signed Messages for Contract Accounts" as a SEP draft in stellar-protocol, and a defined `signMessage` behavior for `C…` addresses in SEP-43 with a single return format. Today that behavior is undefined.
2. **A reference verifier with a published Wasm hash**, deployed on testnet and mainnet by whoever needs it. Admin-less, immutable, one function, audited. Wallets and verifiers recognize it by its hash, so a wallet can render the request as "sign this message", not as an opaque contract call.
3. **A TypeScript library on npm.** `signMessage` and `verifyMessage` work the same for `G…` accounts (SEP-53 underneath) and `C…` accounts (enforcing simulation underneath). Adapters for OpenZeppelin smart accounts, passkey-kit and multisig accounts; a helper for wallets that build the entry themselves; Express and Next.js middleware for servers.
4. **Integrations.** A Stellar Wallets Kit module, pull requests to `stellar/smart-account-kit` and to passkey wallets such as SoroPass, an x402 sign-in-with-x verifier for Stellar and a CAIP-122 `stellar` profile, so Stellar joins EVM and Solana in the cross-chain sign-in ecosystem.
5. **A public conformance suite.** Positive test vectors for every supported account type and negative vectors (removed signer, rotated signer, expired signature, wrong network, wrong domain, tampered message, reused nonce, policy rejection, extra sub-call, cross-account replay, displayed text not matching signed text). Any wallet can run it and prove compatibility.
6. **A live demo and wallet display guidance**, with the signer page and the verifier service on different origins.

What changes for each side

- **Wallet developers** stop throwing on `signMessage` for contract accounts. They call one function; no per-account verification code, no custom envelope.
- **dApp and backend developers** verify a signature from any Stellar account with one call. No transaction, no fee, under two seconds, and the answer reflects the account's rules at that moment.
- **End users with a passkey or multisig wallet** get a "Sign" button that works everywhere: off-chain votes, approvals, sign-in, proof of ownership. They see the exact text they sign. A device that was removed from the account can no longer sign on their behalf.
- **The ecosystem** gets the missing primitive that funded projects already hit: Pollar (SCF #45) errors on C addresses today; Haven (SCF #45) is a passkey smart-account wallet; the three x402 facilitators funded in SCF #45 (Rail402, AgentSmith, Rumble Fish) need a sign-in-with-x profile for Stellar, which x402 ships only for EVM and Solana; the Passkey UI Kit (SoroPass, SCF #44) has no `signMessage` in its funded scope.

A day with C-Sign, once it is done

Alice holds an OpenZeppelin smart account controlled by two passkeys (phone and laptop). A governance dApp asks her to vote off-chain on "Proposal #12". Her wallet shows the text, she confirms with Face ID, and the dApp sends the signed entry to the dApp's server. The server calls `verifyMessage`: it checks the structure, runs an enforcing simulation against a public RPC, and gets `AUTH_OK` back in about one second. No transaction, no fee. The vote is recorded.

A week later Alice loses her phone and removes that passkey from the account. Every signature that phone produced before now fails verification, because the account's current rules no longer include it. The vote the server already recorded stays recorded; what changes is that nobody can produce new signatures with the lost phone.

The same afternoon Alice pays for an x402-metered API with her smart account. The next time she calls it, the server asks her to sign in with x; her wallet signs a C-Sign message, the server verifies it and recognizes her as a returning payer. No new payment, no session cookie, no Stellar-specific code in the x402 server beyond the C-Sign verifier.

Why now

- **SDF wants this solved before SEP-43 is finalized** and has asked for a new SEP as the counterpart of SEP-53.
- **Nobody is building it.** Among the 46 projects funded in SCF #44, the 40 funded in SCF #45, the Türkiye chapter's Instawards cohorts, OpenZeppelin's smart-account roadmap, `stellar/smart-account-kit` and `stellar/passkey-kit`, there is no message-signing standard or verifier for contract accounts (checked again on 28 September 2026).
- **The pieces are in place.** V2 credentials (CAP-71), enforcing simulation in stellar-rpc 23, `SimulationAuthMode` in js-stellar-sdk, and the SEP-45 precedent all exist. C-Sign combines them; it does not need new protocol features.

What is built today

- **Live demo, c-sign-demo.vercel.app.** Create an OpenZeppelin smart account with a passkey, sign in to the site without a transaction, and watch every verification step. Then rotate the key: add a recovery key, remove the passkey, and the first signature turns invalid while the new key signs in.
- **Reference verifier on testnet.** Two independent instances of the same Wasm (none is canonical), both running the pinned hash, deployed by an idempotent script. The Wasm is 1,475 bytes, builds reproducibly on macOS and Linux CI, and has 5 unit tests.
- **TypeScript library (`packages/c-sign`).** Message encoding, entry building, OpenZeppelin signing and `verifyMessage`, which runs the checks of the draft SEP in order and answers `valid`, `invalid` or `inconclusive`. 31 offline unit tests run in CI.
- **25 testnet experiments, all passing, with transaction links (EXPERIMENTS.md).** A valid signature submitted on-chain fails and leaves the account's nonce unused, and it still verifies afterwards; a fake verifier is caught by hash pinning; tampering, another domain, another account, expiry and nonce replay are rejected; after a key rotation the old signature is invalid. Median verification time is about half a second.
- `docs/SPEC.md`: the draft SEP "Signed Messages for Contract Accounts", written to the SEP template, with the verification algorithm, wallet rules, design rationale and security concerns. `docs/SEP-43-CHANGE.md`: the separate wallet-interface proposal.
- `docs/THREAT-MODEL.md`: trust assumptions, threats with the test that covers each, and the gaps the library still has against SPEC v0.0.2.
- **Test vectors (`vectors/c-sign-v1.json`)**: language-neutral vectors for message encoding, entries, OpenZeppelin signing, the verification checks and the wallet's `domain`, checked in CI.
- `docs/ROADMAP.md`: what is done and what comes next: the library update to SPEC v0.0.2, the SEP pull request, wallet integrations and mainnet.

Main risks

Risk	How it is handled
The signer side differs by account type: OpenZeppelin stellar-accounts 0.8.0 changed its auth digest (C-Sign signs both the old and the new one), passkey-kit and smart-account-kit are not drop-in compatible, `signAuthEntry` means different things in different wallets.	One adapter per account type with its own test vectors. OpenZeppelin accounts with Ed25519 and passkey signers are validated on testnet, a 0.8.0 account with an Ed25519 signer, and a minimal non-OpenZeppelin account; passkey-kit and multisig wallets come next; delegated signers (CAP-71) later.
The `AUTH_OK` revert pattern is new on Stellar.	Confirmed on testnet: a submitted signature fails, the on-chain trace shows the account accepting it and the verifier raising `AUTH_OK`, and the nonce stays unused (EXPERIMENTS.md, E2). The same idea was suggested for SEP-45 and set aside only because the server builds entries there. An RP-bound, non-reverting variant is documented as a fallback.
The cheapest answer is "reject `signMessage` on C addresses", and some teams do that today.	The design is on SDF's table; a pull request to `stellar/smart-account-kit` turns substitution risk into partnership.
Off-chain verification trusts the RPC.	Same model as ERC-1271 with `eth_call`; documented, with own-RPC and two-provider cross-check as options.

Links

- Live demo: c-sign-demo.vercel.app
- Repository: github.com/sayweer/c-sign
- Testnet evidence: docs/EXPERIMENTS.md
- Draft specification: docs/SPEC.md · Threat model: docs/THREAT-MODEL.md · Roadmap: docs/ROADMAP.md
- SDF issues: Discussion #2045 (this proposal) · #2027 (first issue) · #1928 (origin)
- Wallet interface change: docs/SEP-43-CHANGE.md
- Standards referenced: SEP-53, SEP-43, SEP-45, CAP-71